

US-001

SAP Integration via Outbox Pattern + IBM MQ

Owner	Tatenda
Sprint	TBD
Estimate	5 days (3 dev · 1 testing · 1 IBM MQ setup)
Type	Story · Backend · Integration
Status	Ready for development
Component	EquipmentChecklist server (ASP.NET Core 8 + EF Core + Postgres)
Issued by	TauGM — Equipment Checklist Programme

1. User story

As a business owner of Belfast Coal Mine, I want every defect raised by an operator's pre-shift checklist to be reliably forwarded to SAP Plant Maintenance, so that the maintenance team works defects from one trusted system (SAP), procurement plans parts from real defect history, and we never lose a defect because of a network blip or SAP downtime.

1.1 Why this matters (business context)

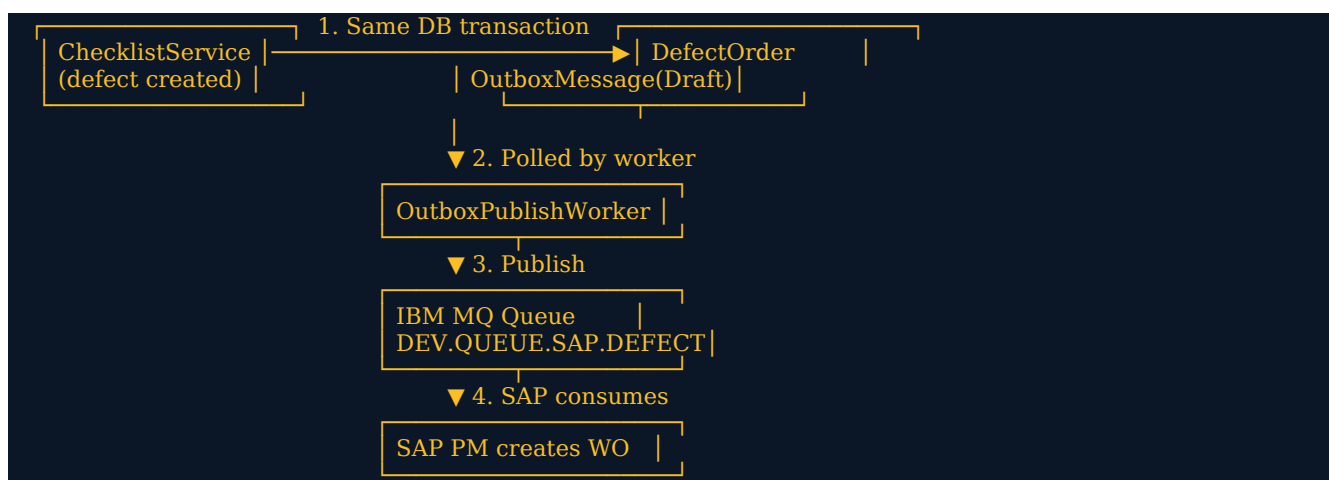
Today an operator finds a wobbly wheel on Truck ADT-04 during their pre-shift check. Our app records it as a DefectOrder. To get that into SAP a human re-types it. The re-typing is where data goes missing: machine numbers get fat-fingered, urgency flags get dropped, half the defects never reach SAP at all because the supervisor is busy.

After this story ships:

- The operator finishes their checklist.
- A row appears in SAP PM within seconds as a Maintenance Order.
- If SAP is down or the network is broken, the message does NOT get lost — it parks in our outbox until SAP is back.
- Every step is auditable end-to-end.

This is the foundation for retiring the "WhatsApp the workshop supervisor" workaround everyone currently uses.

2. Architecture overview



2.1 Two guarantees this architecture provides

- **Atomicity** — The DefectOrder and the OutboxMessage are saved in the SAME EF Core transaction. Either both land or neither does. We can never have a defect on disk with no corresponding outbox message, OR an outbox message with no underlying defect.
- **At-least-once delivery** — The worker only marks an outbox row as Sent AFTER IBM MQ has acknowledged the publish. If the worker crashes mid-publish, the row stays Draft and the next worker pass picks it up. SAP must handle duplicates idempotently — it does, using our DefectOrderId as the idempotency key.

3. Acceptance criteria

Six Gherkin scenarios. All must pass automated tests before this story moves to Done.

Scenario 1 — Defect is written to outbox in the same transaction

Given an operator submits a NO-GO checklist for ADT-04 with a critical brake defect
When ChecklistService.ProcessSubmissionAsync completes successfully
Then a DefectOrder row exists in the database
And exactly one OutboxMessage row exists with:
- AggregateType = "DefectOrder"
- AggregateId = the DefectOrder.Id
- MessageType = "DefectOrderCreated"
- Status = "Draft"
- PayloadJson = the serialised IntegrationDefectPayload
- AttemptCount = 0

Scenario 2 — Background worker publishes Draft messages

Given there are 3 OutboxMessage rows with Status = "Draft"
When the OutboxPublishWorker runs its next pass
Then all 3 are published to IBM MQ queue DEV.QUEUE.SAP.DEFECT
And all 3 rows are updated to Status = "Sent"
And all 3 rows have ProcessedAt set to UtcNow

Scenario 3 — Publish failure is retried, not lost

Given an OutboxMessage with Status = "Draft" and AttemptCount = 0
And IBM MQ is unreachable
When the OutboxPublishWorker tries to publish
Then the row stays at Status = "Draft"
And AttemptCount is incremented to 1
And LastError contains the exception message
And the next worker pass tries again with exponential backoff

Scenario 4 — Permanently failed message lands in DeadLetter

Given an OutboxMessage with AttemptCount = 5 (configurable max retries)
And the publish still fails
Then the row is updated to Status = "DeadLetter"
And an audit event "outbox.dead_lettered" is written
And the row appears on Admin → Outbox Dead Letter page for manual review

Scenario 5 — Atomicity: DB rollback rolls back the outbox too

Given ChecklistService.ProcessSubmissionAsync is mid-transaction
When SaveChangesAsync throws (e.g. DB constraint violation)
Then neither the DefectOrder nor the OutboxMessage row exists
And no message is published to IBM MQ

Scenario 6 — Same DefectOrderId arriving twice is handled

Given the worker publishes DefectOrder #123 but the network drops before marking Sent
And the worker retries on next pass and publishes it again
Then SAP sees two messages with the same idempotency key
And SAP processes the first and ignores the second (idempotent on their side)
And our OutboxMessage eventually reaches Status = "Sent"

4. Technical design

4.1 New table — OutboxMessages

```
CREATE TABLE IF NOT EXISTS "OutboxMessages" (
  "Id"          bigserial PRIMARY KEY,
  "AggregateType" varchar(40) NOT NULL, -- e.g. "DefectOrder"
  "AggregateId"  varchar(80) NOT NULL,  -- e.g. "1234"
  "MessageType"  varchar(80) NOT NULL,  -- e.g. "DefectOrderCreated"
  "PayloadJson"  text          NOT NULL,
  "Status"       varchar(20) NOT NULL,  -- Draft / Sent / DeadLetter
  "AttemptCount" integer       NOT NULL DEFAULT 0,
  "LastError"    varchar(2000) NULL,
  "CreatedAt"    timestamp with time zone NOT NULL,
  "ProcessedAt"  timestamp with time zone NULL,
  "NextAttemptAt" timestamp with time zone NULL
);
CREATE INDEX IF NOT EXISTS "IX_OutboxMessages_Status_NextAttemptAt"
ON "OutboxMessages" ("Status", "NextAttemptAt");
```

Bootstrap via the same ALTER TABLE IF NOT EXISTS pattern we used for AppSettings and the audit hash-chain columns. See Program.cs for prior art.

4.2 Modified — ChecklistService.cs

Where it currently calls `_integration.PublishDefectOrderCreatedAsync(payload)` fire-and-forget, replace with the outbox write happening in the same DbContext transaction:

```
_db.OutboxMessages.Add(new OutboxMessage {
  AggregateType = "DefectOrder",
  AggregateId   = order.Id.ToString(),
  MessageType   = "DefectOrderCreated",
  PayloadJson   = JsonSerializer.Serialize(payload),
  Status        = "Draft",
  CreatedAt     = DateTime.UtcNow,
  NextAttemptAt = DateTime.UtcNow,
  AttemptCount  = 0
});
// SaveChangesAsync was already going to be called for the DefectOrder.
// Both rows land in the same transaction — atomic.
```

4.3 New service — OutboxPublishWorker.cs

A BackgroundService that wakes every 5 seconds, batch-fetches Draft messages whose NextAttemptAt has elapsed, and dispatches them to the configured publisher:

```
var batch = await _db.OutboxMessages
  .Where(m => m.Status == "Draft" && m.NextAttemptAt <= DateTime.UtcNow)
  .OrderBy(m => m.CreatedAt)
  .Take(50)
  .ToListAsync(ct);
```

On success: Status = "Sent", ProcessedAt = UtcNow. On failure: AttemptCount++, LastError = ex.Message, NextAttemptAt = UtcNow + (2^AttemptCount) seconds (exponential backoff capped at 1 hour). After AttemptCount >= 5, set Status = "DeadLetter" and write an audit event.

4.4 New publisher — IbmMqIntegrationPublisher.cs

Replaces the existing SapPmiIntegrationPublisher (which posted directly via HTTP). New implementation publishes to IBM MQ using the official .NET XMS client:

```
// NuGet: IBMXMSDotnetClient (the IBM-supported .NET 8 client)
using IBM.XMS;

var factory = XMSFactoryFactory.GetInstance(XMSC.CT_WMQ);
var cf = factory.CreateConnectionFactory();
cf.SetStringProperty(XMSC.WMQ_HOST_NAME, _hostName); // localhost in dev
cf.SetIntProperty(XMSC.WMQ_PORT, _port); // 1414
cf.SetStringProperty(XMSC.WMQ_CHANNEL, _channel); // DEV.APP.SVRCONN
cf.SetStringProperty(XMSC.WMQ_QUEUE_MANAGER, qManager); // QM1
cf.SetIntProperty(XMSC.WMQ_CONNECTION_MODE, XMSC.WMQ_CM_CLIENT);
cf.SetStringProperty(XMSC.USERID, _user); // app
cf.SetStringProperty(XMSC.PASSWORD, _password); // passw0rd in dev
```

```

using var conn    = cf.CreateConnection();
using var session = conn.CreateSession(false, AcknowledgeMode.AutoAcknowledge);
using var dest    = session.CreateQueue("queue://" + _queueName);
using var producer = session.CreateProducer(dest);

var msg = session.CreateTextMessage(payloadJson);
msg.SetStringProperty("messageType", messageType);
msg.SetStringProperty("idempotencyKey", defectOrderId.ToString());
producer.Send(msg);

```

4.5 Configuration keys (seeded into AppSettings)

Key	Default	Description
Mq.Enabled	false	Master switch. False → outbox rows pile up but worker doesn't publish. Useful when SAP isn't ready yet.
Mq.HostName	localhost	IBM MQ broker hostname.
Mq.Port	1414	IBM MQ TCP port.
Mq.QueueManager	QM1	Queue manager name.
Mq.Channel	DEV.APP.SVRCONN	Server-connection channel.
Mq.QueueName	DEV.QUEUE.SAP.DEFECT	Queue to publish to.
Mq.User	app	MQ user.
Mq.Password	(secret)	MQ password — IsSecret = true.
Mq.MaxRetries	5	Outbox attempts before DeadLetter.
Mq.WorkerIntervalSeconds	5	How often the worker polls Draft messages.

5. Implementation steps (5 days, in order)

Day 1 — Local IBM MQ Docker setup

1. Install Docker Desktop if not already (<https://docs.docker.com/desktop/install/windows-install/>).
2. Pull the IBM MQ developer image (free, fully featured for dev use): `docker pull icr.io/ibm-messaging/mq:latest`
3. Start the container with persistent storage (see the `docker-compose.yml` in §6 — preferred) or run it directly via the `docker run` shown below.
4. Verify it's up: open `https://localhost:9443/ibmmq/console`, accept the self-signed cert, log in as `admin / passw0rd`. Queue Manager QM1 should be running.
5. Create the application queue: Queues → Create → Local, Name `DEV.QUEUE.SAP.DEFECT`, save.
6. Commit `docker-compose.yml` at repo root so the next dev can run ``docker compose up -d`` instead of remembering commands.

Day 2 — Outbox table + entity + write path

7. Create `Models/OutboxMessage.cs` (POCO matching the schema in §4.1).
8. Register the `DbSet` in `ApplicationDbContext`.
9. Bootstrap the table in `Program.cs` using `ALTER TABLE IF NOT EXISTS` pattern (see audit hash-chain code as reference).
10. Modify `ChecklistService.cs` to write to the outbox in the same transaction as `DefectOrder` creation. Remove the existing fire-and-forget call to `_integration.PublishDefectOrderCreatedAsync`.
11. Write a unit test that calls `ProcessSubmissionAsync` and asserts both rows exist after `SaveChangesAsync`, and neither exists if the save throws.
12. Manually test: submit a NO-GO from the mobile app, query the DB, confirm the `OutboxMessage` row appears with `Status='Draft'`.

Day 3 — Publisher + background worker

13. Add NuGet package `IBMXMSDotnetClient` to `EquipmentChecklist.csproj`.
14. Create `Services/Integrations/IbmMqIntegrationPublisher.cs` as outlined in §4.4.
15. Create `Services/Integrations/OutboxPublishWorker.cs` as a `BackgroundService` implementing the polling + retry + backoff logic from §4.3.
16. Register both services in `Program.cs` DI graph: `AddScoped<IIntegrationPublisher, IbmMqIntegrationPublisher>()` and `AddHostedService<OutboxPublishWorker>()`.
17. Seed the `Mq.*` config keys in `SeedAppSettingsAsync`.
18. Manually test end-to-end: submit NO-GO → wait 10 seconds → check `OutboxMessage` row shows `Status='Sent'` → open MQ console and verify the message is on `DEV.QUEUE.SAP.DEFECT`.

Day 4 — Admin UI + dead-letter handling

19. Create `/Admin/Outbox` controller action + Razor view: counts of Draft / Sent (last 24h) / DeadLetter.
20. Add a table of recent 50 messages with `Id`, `Status`, `AttemptCount`, `CreatedAt`, `LastError`.
21. Add a "Retry now" button on DeadLetter rows (resets `Status=Draft`, `AttemptCount=0`).
22. Add a "Purge sent older than 30 days" button.
23. Surface the link in `_Layout.cshtml` sidebar under "Integrations".

Day 5 — Testing + handover

24. Write integration tests (against a Testcontainers IBM MQ instance): happy path, MQ down → backoff, permanent failure → DeadLetter, atomic rollback.
25. Update `Device_Setup_Runbook.docx` with the new "Integrations → Outbox" admin page.
26. Demo to stakeholders with SAP basis team present.

6. Docker setup — IBM MQ on a dev laptop

Save this as docker-compose.yml at repo root. Any dev can then run `docker compose up -d` and have IBM MQ running locally in 30 seconds:

```
version: '3.8'
services:
  ibm-mq:
    image: icr.io/ibm-messaging/mq:latest
    container_name: ibm-mq-dev
    environment:
      LICENSE: accept
      MQ_QM_NAME: QM1
      MQ_APP_PASSWORD: passw0rd
      MQ_ADMIN_PASSWORD: passw0rd
    ports:
      - "1414:1414" # MQ traffic
      - "9443:9443" # web console (HTTPS)
    volumes:
      - qm1data:/mnt/mqm
    restart: unless-stopped

volumes:
  qm1data:
```

Useful commands:

```
docker compose up -d # start MQ
docker compose logs -f # tail logs
docker compose down # stop MQ
docker compose down -v # stop AND wipe data (clean slate)
```

Web console URL: <https://localhost:9443/ibmmq/console> — log in as admin / passw0rd. The IBM developer licence is free for non-production use; the same image goes to paid licensing in production (factored into Risks §9).

7. Definition of Done

- All 6 acceptance scenarios pass automated tests.
- docker-compose.yml checked in at repo root.
- IBM MQ developer container running locally produces visible messages on DEV.QUEUE.SAP.DEFECT after a NO-GO submission.
- /Admin/Outbox page exists and shows real counts.
- Dead-letter retry button works.
- Audit event "outbox.dead_lettered" fires on permanent failures.
- Code review by senior dev signed off.
- Demo recorded (screen capture) and shared with SAP basis team.
- This document moved from UserStories/ to UserStories/Completed/ with PR link appended.

8. Open questions for the team

27. SAP basis team — should we send OData JSON or IDoc XML inside the MQ message? Default in this story: JSON. If they want IDoc, change PayloadJson to PayloadXml and adjust IbmMqIntegrationPublisher to serialise differently. Same outbox, different payload shape.
28. What's our MaxRetries for production? Default 5 = ~30 minutes of retries. SHE may want longer (24 hours) so an overnight SAP outage doesn't dead-letter anything.
29. Do we purge Sent rows after 30 days or keep them indefinitely? Recommendation: keep 90 days, purge older.

9. Risks

- **IBM MQ licence cost in production** — The dev image is free; production licences are per-PVU and start around USD 5 000/year for a small queue manager. Confirm corporate IT has a licence before going to prod — OR consider RabbitMQ as a free alternative. The outbox pattern works identically against either; only IbmMqIntegrationPublisher changes.
- **MQ admin skills are scarce in SA mining** — Most SAP basis teams know MQ administration; if yours doesn't, factor in training time.

- **Atomicity gotcha** — The outbox insert **MUST** happen inside the same `_db` transaction as the `DefectOrder` insert. If a future refactor moves them into separate `SaveChangesAsync` calls, atomicity is silently broken. Add a unit test that asserts both rows are in the same transaction.

10. References

- `Services/Integrations/IIntegrationPublisher.cs` — existing interface (we re-use it).
- `Services/Integrations/SapPmIntegrationPublisher.cs` — existing HTTP scaffold (will be replaced by `IbmMqIntegrationPublisher`).
- `Services/AuditService.cs` — pattern for `BackgroundService` with `ChainLock` (similar concurrency considerations).
- `Program.cs` → `SeedAppSettingsAsync` — pattern for new config keys.
- IBM MQ Developer docs: <https://developer.ibm.com/components/ibm-mq/>
- IBM XMS .NET client docs: <https://www.ibm.com/docs/en/ibm-mq/9.3?topic=mq-using-net-xms>